LegalEase

Project Documentation

Complete contents of the LegalEase project, one section per file.

Project structure

```
README.md
.env
.env.example
.gitignore
config.py
conftest.py
requirements-dev.txt
requirements.txt
run.bat
run.sh
.streamlit/config.toml
.vscode/launch.json
Image/Logo.png
Image/inverseLogo.png
ai_core/__init__.py
ai_core/gemini_generator.py
ai_core/generator.py
ai_core/text_utils.py
frontend/app.py
legalEaseAPI/__init__.py
legalEaseAPI/main.py
legalEaseAPI/routes.py
scripts/check_gemini.py
scripts/create_logos.py
tests/test_api.py
tests/test_formatting.py
tests/test_gemini_generator.py
```

README.md

````
# LegalEase - AI-Powered Legal Document Generator

Streamlit frontend + FastAPI backend + Google Gemini. Generates contracts, NDAs and leases from
a few inputs, lets you edit the result, and exports **.txt / .docx / .pdf** with branding.

```
LegalEase/
??? ai_core/
?   ??? gemini_generator.py   # Gemini prompt + API call (model fallback, error handling)
?   ??? generator.py          # parse text -> HTML preview, DOCX, PDF
?   ??? text_utils.py         # sanitize_text, split_terms, safe_filename
??? legalEaseAPI/
?   ??? main.py               # FastAPI app, "/" route, includes router
?   ??? routes.py             # DocumentRequest model, POST /generate, GET /health
??? frontend/app.py           # Streamlit UI
??? Image/                    # Logo.png (documents), inverseLogo.png (dark web UI)
??? scripts/                  # create_logos.py, check_gemini.py
??? tests/                    # unit + API tests
??? config.py  .env  requirements.txt  run.bat  run.sh
??? .streamlit/config.toml    # dark theme
```

## 1. Setup in VS Code

**Prerequisites:** Python 3.10+ and VS Code with the *Python* extension. A free Gemini API key from
<https://aistudio.google.com/apikey>.

1. Unzip the project, then in VS Code: **File > Open Folder...** and choose `LegalEase`.
2. Open a terminal: **Terminal > New Terminal**.
3. Create and activate a virtual environment:

   | OS | Commands |
   |---|---|
   | Windows (PowerShell) | `python -m venv venv` then `venv\Scripts\activate` |
   | macOS / Linux | `python3 -m venv venv` then `source venv/bin/activate` |

   If PowerShell blocks activation, run `Set-ExecutionPolicy -Scope Process Bypass` first.
4. Install dependencies: `pip install -r requirements.txt`
5. In VS Code press `Ctrl+Shift+P` > **Python: Select Interpreter** > pick the one inside `venv`.
6. Open `.env` and replace the placeholder:
   ```
   GEMINI_API_KEY=AIza...your real key...
   ```

## 2. Run
**Easiest:** `run.bat` (Windows) or `./run.sh` (macOS/Linux) starts both servers.

**Manual (two terminals, both with the venv activated):**

```bash
# Terminal 1 - backend  -> http://127.0.0.1:8000  (API docs at /docs)
uvicorn legalEaseAPI.main:app --reload

# Terminal 2 - frontend -> http://localhost:8501
streamlit run frontend/app.py
```

Or use **Run and Debug** (`Ctrl+Shift+D`) > *LegalEase (API + UI)* (config in
    `.vscode/launch.json`).

## 3. Use it

1. Document Type: `Freelance Work Contract`
2. Parties: `Jane Doe (Service Provider), TechNova Inc. (Client)`
3. Terms (semicolon-separated): `Work must be delivered by May 15, 2025; Payment within 7 days of
    invoice; The client retains intellectual property rights; Confidentiality must be maintained at
    all times`
````

```
4. Effective Date: `April 15, 2025` > **Generate Document**
5. Read the dark preview, click **Click to Edit Document** to change the wording, then download
    `.TXT`, `.DOCX` or `.PDF`.

## 4. Test

```bash
# a) Check key + model (live call to Gemini)
python scripts/check_gemini.py

# b) Backend alone
curl http://127.0.0.1:8000/health
curl -X POST http://127.0.0.1:8000/generate -H "Content-Type: application/json" \
  -d "{\"document_type\":\"NDA\",\"parties\":\"A (Discloser), B (Recipient)\",\"terms\":\"Keep
    secret; 2 years\",\"dates\":\"May 1, 2026\"}"
# (or use the interactive page at http://127.0.0.1:8000/docs)

# c) Automated tests (no API key or internet needed)
pip install -r requirements-dev.txt
pytest -v
```
Manual checklist: preview renders headings/bullets - edit changes the preview - the DOCX has logo,
    terms table
and footer - the PDF has logo header and footer on every page - empty fields show a warning -
    stopping the
backend shows a clear "cannot reach backend" message.

5. Troubleshooting

Symptom	Fix
"GEMINI_API_KEY is not set" / "key was rejected"	Put a valid key in `.env`, restart uvicorn (it
reads `.env` at start).
"All Gemini models failed ... 404 / not found"	Set `GEMINI_MODEL=` in `.env` to a model your key
can use (see <https://ai.google.dev/gemini-api/docs/models>).
429 / quota errors	Free-tier rate limit; wait a minute or switch model.
"Backend offline" in sidebar	Start uvicorn from the project root (the folder containing
`config.py`).
`ModuleNotFoundError: ai_core`	Run commands from the project root, not from inside a sub-folder.
Non-Latin characters show as `?` in the PDF	The built-in PDF font is Latin-1 (? is written as
"INR"). Use DOCX/TXT for other scripts.

6. Notes

* **Model:** the brief names `gemini-1.5-pro`, which Google has retired. `GEMINI_MODEL` defaults to
    the
  `gemini-flash-latest` alias, with automatic fallback to the models in `GEMINI_FALLBACK_MODELS`.
  Model names change often, so `.env` is the one place to update.
* **SDK:** uses `google-genai`; falls back to `google-generativeai` (from the brief) if only that is
     installed.
* **PDF library:** `fpdf2` (the maintained successor; still imported as `fpdf`).
* **Privacy:** inputs go only to your backend and Gemini; nothing is stored. Never commit `.env`.
* **Disclaimer:** output is an AI draft, not legal advice. Have a lawyer review before signing.
* **Deployment (optional):** backend `uvicorn legalEaseAPI.main:app --host 0.0.0.0 --port $PORT`
  (Render/Railway/Fly.io, set `GEMINI_API_KEY` as an env var); frontend on Streamlit Community Cloud
     with
  `BACKEND_URL` set to the deployed backend URL.
```

.env

Omitted - contains secrets. See .env.example for the required variables.

.env.example

```
# Copy this file to ".env" and paste your key. Get one free at https://aistudio.google.com/apikey
GEMINI_API_KEY=your_gemini_api_key_here
```

```
# Optional overrides
# GEMINI_MODEL=gemini-flash-latest
# GEMINI_FALLBACK_MODELS=gemini-3-flash-preview,gemini-2.5-flash,gemini-2.5-pro
# BACKEND_URL=http://localhost:8000
# COMPANY_NAME=LegalEase Inc.
# CONTACT_EMAIL=contact@legalease.com
```

.gitignore

```
venv/
.venv/
__pycache__/
*.pyc
.env
.pytest_cache/
*.egg-info/
.DS_Store
```

config.py

```
"""Central configuration for LegalEase.

All values can be overridden through environment variables or the .env file
that sits next to this file.
"""
import os
from pathlib import Path

from dotenv import load_dotenv

BASE_DIR = Path(__file__).resolve().parent
load_dotenv(BASE_DIR / ".env")

# ---------------------------------------------------------------- Gemini / AI
GEMINI_API_KEY: str = os.getenv("GEMINI_API_KEY", "").strip()

# The project brief names "gemini-1.5-pro", but Google has retired the 1.5
# family.  The default below is an always-current alias; change it in .env if
# you want a specific model (e.g. gemini-2.5-pro, gemini-3-flash-preview ...).
GEMINI_MODEL: str = os.getenv("GEMINI_MODEL", "gemini-flash-latest").strip()

# If GEMINI_MODEL is unavailable for your key, these are tried in order.
GEMINI_FALLBACK_MODELS = [
    m.strip()
    for m in os.getenv(
        "GEMINI_FALLBACK_MODELS",
        "gemini-3-flash-preview,gemini-2.5-flash,gemini-2.5-pro",
    ).split(",")
    if m.strip()
]

# ---------------------------------------------------------------- Backend / UI
BACKEND_URL: str = os.getenv("BACKEND_URL", "http://localhost:8000").rstrip("/")
REQUEST_TIMEOUT: int = int(os.getenv("REQUEST_TIMEOUT", "180"))  # seconds

# ---------------------------------------------------------------- Branding
COMPANY_NAME: str = os.getenv("COMPANY_NAME", "LegalEase Inc.")
CONTACT_EMAIL: str = os.getenv("CONTACT_EMAIL", "contact@legalease.com")
FOOTER_TEXT: str = f"{COMPANY_NAME} | {CONTACT_EMAIL} | All Rights Reserved."
DOC_FONT: str = "Times New Roman"

IMAGE_DIR = BASE_DIR / "Image"
DOC_LOGO_PATH = IMAGE_DIR / "Logo.png"          # dark logo for white paper
WEB_LOGO_PATH = IMAGE_DIR / "inverseLogo.png"   # light logo for the dark web UI
```

conftest.py

```
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))
```

requirements-dev.txt

```
-r requirements.txt
pytest>=8.0
httpx>=0.27
```

requirements.txt

```
fastapi>=0.110
uvicorn[standard]>=0.29
pydantic>=2.6
streamlit>=1.35
python-docx>=1.1
fpdf2>=2.7.9
Pillow>=10.1
requests>=2.31
python-dotenv>=1.0
google-genai>=1.0
```

run.bat

```
@echo off
REM Starts the FastAPI backend (new window) and the Streamlit frontend (Windows).
cd /d %~dp0
start "LegalEase API" cmd /k "venv\Scripts\activate && uvicorn legalEaseAPI.main:app --host
    127.0.0.1 --port 8000 --reload"
timeout /t 3 >nul
call venv\Scripts\activate
streamlit run frontend\app.py
```

run.sh

```
#!/usr/bin/env bash
# Starts the FastAPI backend and the Streamlit frontend (macOS / Linux / Git-Bash).
set -e
cd "$(dirname "$0")"
if [ -f venv/bin/activate ]; then source venv/bin/activate; fi
if [ -f venv/Scripts/activate ]; then source venv/Scripts/activate; fi

uvicorn legalEaseAPI.main:app --host 127.0.0.1 --port 8000 --reload &
API_PID=$!
trap 'kill $API_PID 2>/dev/null || true' EXIT INT TERM
sleep 2
streamlit run frontend/app.py
```

.streamlit/config.toml

```
[theme]
base = "dark"

[browser]
gatherUsageStats = false
```

.vscode/launch.json

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "LegalEase API (FastAPI)",
      "type": "debugpy",
      "request": "launch",
      "module": "uvicorn",
      "args": ["legalEaseAPI.main:app", "--reload", "--port", "8000"],
      "cwd": "${workspaceFolder}",
      "console": "integratedTerminal"
    },
    {
      "name": "LegalEase UI (Streamlit)",
      "type": "debugpy",
      "request": "launch",
      "module": "streamlit",
      "args": ["run", "frontend/app.py"],
      "cwd": "${workspaceFolder}",
      "console": "integratedTerminal"
    }
  ],
  "compounds": [
    { "name": "LegalEase (API + UI)", "configurations": ["LegalEase API (FastAPI)", "LegalEase UI
    (Streamlit)"] }
  ]
}
```

Image/Logo.png

Binary image file, not shown.

Image/inverseLogo.png

Binary image file, not shown.

ai_core/__init__.py

ai_core/gemini_generator.py

```
"""Gemini integration: turns the user's inputs into a full legal document.

Supports the current ``google-genai`` SDK and, if that is not installed, the
older ``google-generativeai`` package named in the project brief.
"""
from __future__ import annotations

import logging
from typing import List, Optional

import config
from ai_core.text_utils import split_terms

logger = logging.getLogger(__name__)


class GeminiConfigError(RuntimeError):
    """Missing API key or SDK."""
class GeminiGenerationError(RuntimeError):
    """Gemini could not produce a document."""
```

```
SYSTEM_INSTRUCTION = (
    "You are a senior legal drafter. You write clear, professional, well-structured "
    "legal documents in plain text for individuals and small businesses. You never "
    "invent facts: anything unknown is written as a [Bracketed Placeholder]. You do "
    "not add commentary, explanations or disclaimers outside the document itself."
)


def build_prompt(document_type: str, parties: str, terms: str, dates: str) -> str:
    """Build the structured prompt sent to Gemini."""
    term_items = split_terms(terms)
    if term_items:
        terms_block = "\n".join(f"{i}. {t}" for i, t in enumerate(term_items, 1))
    else:
        terms_block = "(none supplied - use standard, balanced terms for this document type)"

    return (
        f"Generate a comprehensive legal document titled '{document_type}'.\n\n"
        f"Involved parties: {parties}\n"
        f"Effective date: {dates}\n"
        f"Terms and conditions that MUST each be reflected as clearly drafted clauses:\n"
        f"{terms_block}\n\n"
        "Drafting requirements:\n"
        "- Ensure formal legal structure with multiple sections and legal clauses.\n"
        "- The first line is the document title in capital letters.\n"
        "- Include: a preamble/recitals, definitions where useful, the obligations of "
        "each party, term and termination, confidentiality, dispute resolution, "
        "governing law, entire agreement, severability, amendments, and a signature "
        "block with signature lines for every party - adapted to what this document type needs.\n"
        "- Number the sections like '1. Services:' with each heading on its own line.\n"
        "- Write sub-clauses on separate lines starting 'a)', 'b)', 'c)'.\n"
        "- Use [Bracketed Placeholders] for any missing detail (addresses, amounts, "
        "jurisdiction). Do not invent names, numbers or dates.\n"
        "- Output PLAIN TEXT only: no markdown symbols (#, *, **, backticks) and no tables.\n"
        "- Output only the document, with nothing before or after it."
    )


class GeminiDocumentGenerator:
    """Wrapper around the Gemini API used by the FastAPI routes."""

    def __init__(
        self,
        model_name: Optional[str] = None,
        api_key: Optional[str] = None,
        fallback_models: Optional[List[str]] = None,
    ) -> None:
        self.model_name = model_name or config.GEMINI_MODEL
        self.api_key = api_key if api_key is not None else config.GEMINI_API_KEY
        self.fallback_models = (
            list(fallback_models) if fallback_models is not None else
    list(config.GEMINI_FALLBACK_MODELS)
        )
        self.active_model: Optional[str] = None  # model that last succeeded
        self._sdk: Optional[str] = None
        self._client = None
        self._legacy = None

    # ------------------------------------------------------------------ setup
    def _ensure_client(self) -> None:
        if self._sdk:
            return
        if not self.api_key or self.api_key.startswith("your_"):
            raise GeminiConfigError(
                "GEMINI_API_KEY is not set. Add your key to the .env file "
                "(get one at https://aistudio.google.com/apikey) and restart the backend."
            )
        try:
            from google import genai  # new SDK
            self._client = genai.Client(api_key=self.api_key)
```

```
        self._sdk = "genai"
        return
    except ImportError:
        pass
    try:
        import google.generativeai as legacy  # SDK named in the brief

        legacy.configure(api_key=self.api_key)
        self._legacy = legacy
        self._sdk = "legacy"
    except ImportError as exc:
        raise GeminiConfigError(
            "No Gemini SDK found. Run: pip install google-genai"
        ) from exc

# ------------------------------------------------------------- generation
def _call_model(self, model: str, prompt: str) -> str:
    if self._sdk == "genai":
        from google.genai import types

        response = self._client.models.generate_content(
            model=model,
            contents=prompt,
            config=types.GenerateContentConfig(
                system_instruction=SYSTEM_INSTRUCTION,
                temperature=0.3,
                max_output_tokens=16000,
            ),
        )
    else:
        gm = self._legacy.GenerativeModel(model_name=model,
system_instruction=SYSTEM_INSTRUCTION)
        response = gm.generate_content(
            prompt, generation_config={"temperature": 0.3, "max_output_tokens": 16000}
        )
    try:
        text = response.text
    except Exception:  # blocked / empty candidates raise in some SDK versions
        text = None
    return (text or "").strip()
@staticmethod
def _is_auth_error(exc: Exception) -> bool:
    msg = str(exc).lower()
    return any(s in msg for s in ("api key not valid", "api_key_invalid", "api key expired",
                                  "invalid api key", "unauthenticated"))

def _candidates(self) -> List[str]:
    ordered = ([self.active_model] if self.active_model else []) + [self.model_name] +
self.fallback_models
    seen, result = set(), []
    for m in ordered:
        if m and m not in seen:
            seen.add(m)
            result.append(m)
    return result

def generate_document(self, document_type: str, parties: str, terms: str, dates: str) -> str:
    """Return the generated legal document as plain text."""
    self._ensure_client()
    prompt = build_prompt(document_type, parties, terms, dates)

    errors = []
    for model in self._candidates():
        try:
            text = self._call_model(model, prompt)
            if not text:
                raise GeminiGenerationError("empty response (possibly blocked by safety
filters)")
            self.active_model = model
            logger.info("Document generated with model %s", model)
```

```
                return text
            except Exception as exc:  # noqa: BLE001 - we want to try the next model
                if self._is_auth_error(exc):
                    raise GeminiConfigError(
                        "The Gemini API key was rejected. Check GEMINI_API_KEY in your .env file."
                    ) from exc
                short = " ".join(str(exc).split())[:220]
                logger.warning("Model %s failed: %s", model, short)
                errors.append(f"{model}: {short}")

        raise GeminiGenerationError(
            "All Gemini models failed. Set GEMINI_MODEL in .env to a model your key can use. "
            "Details -> " + " | ".join(errors)
        )
```

ai_core/generator.py

```
"""Document formatting: parse AI text and export it as HTML preview, .docx and .pdf.

Public API
----------
sanitize_text(text)                       -> clean plain text
format_html_preview(text, doc_type="")    -> HTML string (inline-styled blocks)
format_docx(text, doc_type, ...)          -> bytes of a .docx file
format_pdf(text, doc_type, ...)           -> bytes of a .pdf file
"""
from __future__ import annotations

import html
import io
import re
from pathlib import Path
from typing import List, Optional, Sequence, Tuple

import config
from ai_core.text_utils import safe_filename, sanitize_text, split_terms  # noqa: F401 (re-exported)

Block = Tuple[str, str]  # (kind, text) kind: heading | bullet | sub | para | rule

# ---------------------------------------------------------------------- parsing
_NUMBERED = re.compile(r"^\d+(?:\.\d+)*[.)]\s+\S")
_NUM_PREFIX = re.compile(r"^\d+(?:\.\d+)*[.)]\s+")
_ARTICLE = re.compile(r"^(?:ARTICLE|SECTION|CLAUSE|SCHEDULE)\s+[\dIVXLC]+\b", re.I)
_BULLET = re.compile(r"^[-+]\s+(.+)")
_SUBCLAUSE = re.compile(r"^(?:\([a-zA-Z0-9]{1,4}\)|[a-zA-Z]\)|[ivxlc]{1,4}\))\s+\S")
_RULE = re.compile(r"^[_=\-]{4,}$")


def _is_heading(line: str) -> bool:
    if len(line) > 90:
        return False
    if _NUMBERED.match(line):
        body = _NUM_PREFIX.sub("", line)
        return len(body) <= 70 and "." not in body.rstrip(".:")
    if _ARTICLE.match(line):
        return True
    letters = [c for c in line if c.isalpha()]
    if len(letters) >= 3 and line.isupper():
        return True
    return line.endswith(":") and len(line.split()) <= 8
def _classify(line: str) -> Block:
    if _RULE.match(line):
        return ("rule", line)
    m = _BULLET.match(line)
    if m:
        return ("bullet", m.group(1).strip())
    if _SUBCLAUSE.match(line):
        return ("sub", line)
```

```
    if _is_heading(line):
        return ("heading", line)
    return ("para", line)


def _norm(s: str) -> str:
    return re.sub(r"[^a-z0-9]+", " ", s.lower()).strip()


def _title_from_first_line(line: str, doc_type: str) -> bool:
    if len(line) > 100 or line.endswith((".", ":", ";", ",")):
        return False
    return line.isupper() or (bool(_norm(doc_type)) and _norm(doc_type) in _norm(line))


def parse_document(text: str, doc_type: str = "") -> Tuple[str, List[Block]]:
    """Return (title, blocks). The title is taken from the AI text when present."""
    lines = [ln.strip() for ln in sanitize_text(text).split("\n")]
    lines = [ln for ln in lines if ln]
    title = (doc_type or "Legal Document").strip()
    if lines and _title_from_first_line(lines[0], doc_type):
        title = lines.pop(0)
    return title, [_classify(ln) for ln in lines]


def _detail_rows(doc_type: str, parties: str, effective_date: str) -> List[Tuple[str, str]]:
    rows = []
    if doc_type:
        rows.append(("Document Type", doc_type.strip()))
    if effective_date:
        rows.append(("Effective Date", effective_date.strip()))
    if parties:
        rows.append(("Parties", re.sub(r"\s*\n\s*", "; ", parties.strip())))
    return rows


# ---------------------------------------------------------------- HTML preview
def _esc(s: str) -> str:
    # '$' is escaped so Streamlit's markdown does not treat "$5 ... $6" as LaTeX.
    return html.escape(s).replace("$", "&#36;")


def format_html_preview(text: str, doc_type: str = "") -> str:
    """Convert the document to inline-styled HTML blocks (no blank lines)."""
    title, blocks = parse_document(text, doc_type)
    out = [f"<h3 style='text-align:center;margin:0 0 18px;letter-spacing:.5px;'>{_esc(title)}</h3>"]
    i = 0
    while i < len(blocks):
        kind, txt = blocks[i]
        if kind == "bullet":
            items = []
            while i < len(blocks) and blocks[i][0] == "bullet":
                items.append(f"<li style='margin:2px 0;'>{_esc(blocks[i][1])}</li>")
                i += 1
            out.append(f"<ul style='margin:4px 0 12px 22px;padding:0;'>{''.join(items)}</ul>")
            continue
        if kind == "heading":
            out.append(f"<h4 style='margin:20px 0 6px;color:#f5c542;font-
    size:1.02rem;'>{_esc(txt)}</h4>")
        elif kind == "sub":
            out.append(f"<p style='margin:3px 0 3px 26px;'>{_esc(txt)}</p>")
        elif kind == "rule":
            out.append("<div style='margin:26px 0 4px;width:240px;border-bottom:1px solid
    #94a3b8;'></div>")
        else:
            out.append(f"<p style='margin:0 0 10px;text-align:justify;'>{_esc(txt)}</p>")
        i += 1
    return "".join(out)
# ---------------------------------------------------------------------- DOCX
def _shade(cell, fill_hex: str) -> None:
```

```
    from docx.oxml import OxmlElement
    from docx.oxml.ns import qn

    shd = OxmlElement("w:shd")
    shd.set(qn("w:val"), "clear")
    shd.set(qn("w:color"), "auto")
    shd.set(qn("w:fill"), fill_hex)
    cell._tc.get_or_add_tcPr().append(shd)
def _add_table(doc, headers: Sequence[str], rows: Sequence[Sequence[str]], widths: Sequence[float]):
    from docx.enum.table import WD_TABLE_ALIGNMENT
    from docx.shared import Inches

    table = doc.add_table(rows=1, cols=len(headers))
    table.style = "Table Grid"
    table.alignment = WD_TABLE_ALIGNMENT.CENTER
    for i, head in enumerate(headers):
        cell = table.rows[0].cells[i]
        cell.text = ""
        cell.paragraphs[0].add_run(head).bold = True
        _shade(cell, "D9E2F3")
    for row in rows:
        cells = table.add_row().cells
        for i, val in enumerate(row):
            cells[i].text = str(val)
    for r in table.rows:
        for i, w in enumerate(widths):
            r.cells[i].width = Inches(w)
    return table


def _add_page_number(paragraph, font_size) -> None:
    from docx.oxml import OxmlElement
    from docx.oxml.ns import qn

    run = paragraph.add_run()
    run.font.size = font_size
    run.italic = True
    for kind, text in (("begin", None), (None, "PAGE"), ("end", None)):
        if kind:
            el = OxmlElement("w:fldChar")
            el.set(qn("w:fldCharType"), kind)
        else:
            el = OxmlElement("w:instrText")
            el.set(qn("xml:space"), "preserve")
            el.text = text
        run._r.append(el)


def format_docx(
    text: str,
    doc_type: str = "Legal Document",
    terms: str = "",
    parties: str = "",
    effective_date: str = "",
) -> bytes:
    """Build a Word document: logo, title, details + terms tables, body, footer."""
    from docx import Document
    from docx.enum.text import WD_ALIGN_PARAGRAPH
    from docx.shared import Inches, Pt

    title, blocks = parse_document(text, doc_type)
    font = config.DOC_FONT

    doc = Document()
    section = doc.sections[0]
    section.left_margin = section.right_margin = Inches(1)
    section.top_margin = section.bottom_margin = Inches(0.9)

    normal = doc.styles["Normal"]
    normal.font.name = font
```

```
    normal.font.size = Pt(11)
    normal.element.get_or_add_rPr().get_or_add_rFonts().set(
        "{http://schemas.openxmlformats.org/wordprocessingml/2006/main}eastAsia", font
    )
    for name in ("List Bullet",):
        doc.styles[name].font.name = font

    # Logo (front page)
    logo = Path(config.DOC_LOGO_PATH)
    if logo.exists():
        p = doc.add_paragraph()
        p.alignment = WD_ALIGN_PARAGRAPH.CENTER
        p.add_run().add_picture(str(logo), width=Inches(2.0))

    # Title
    p = doc.add_paragraph()
    p.alignment = WD_ALIGN_PARAGRAPH.CENTER
    p.paragraph_format.space_after = Pt(12)
    run = p.add_run(title)
    run.bold = True
    run.font.size = Pt(16)

    # Key details + auto-generated terms table
    details = _detail_rows(doc_type, parties, effective_date)
    if details:
        _add_table(doc, ["Key Details", ""], details, [1.6, 4.9])
        doc.add_paragraph()
    term_items = split_terms(terms)
    if term_items:
        _add_table(doc, ["No.", "Terms & Conditions"], [(str(i), t) for i, t in
    enumerate(term_items, 1)], [0.6, 5.9])
        doc.add_paragraph()

    # Body
    for kind, txt in blocks:
        if kind == "heading":
            para = doc.add_paragraph()
            para.paragraph_format.space_before = Pt(10)
            para.paragraph_format.space_after = Pt(4)
            para.paragraph_format.keep_with_next = True
            para.add_run(txt).bold = True
        elif kind == "bullet":
            para = doc.add_paragraph(txt, style="List Bullet")
        elif kind == "sub":
            para = doc.add_paragraph(txt)
            para.paragraph_format.left_indent = Inches(0.4)
            para.paragraph_format.space_after = Pt(3)
        elif kind == "rule":
            para = doc.add_paragraph("_" * 32)
            para.paragraph_format.space_before = Pt(18)
            para.paragraph_format.space_after = Pt(0)
        else:
            para = doc.add_paragraph(txt)
            para.alignment = WD_ALIGN_PARAGRAPH.JUSTIFY
            para.paragraph_format.space_after = Pt(6)

    # Footer with page number (every page)
    fp = section.footer.paragraphs[0]
    fp.alignment = WD_ALIGN_PARAGRAPH.CENTER
    r = fp.add_run(f"{config.FOOTER_TEXT}  |  Page ")
    r.font.size = Pt(8)
    r.italic = True
    _add_page_number(fp, Pt(8))

    buf = io.BytesIO()
    doc.save(buf)
    return buf.getvalue()


# ---------------------------------------------------------------------- PDF
```

```
_PDF_MAP = {"\u20b9": "INR ", "\u20ac": "EUR ", "\u2122": "(TM)", "\u2713": "v"}
def _pdf_safe(text: str) -> str:
    """The built-in PDF fonts are Latin-1 only; map or replace anything else."""
    for old, new in _PDF_MAP.items():
        text = text.replace(old, new)
    return text.encode("latin-1", "replace").decode("latin-1")


def _make_pdf_class():
    from fpdf import FPDF
    from fpdf.enums import XPos, YPos

    class LegalPDF(FPDF):
        def __init__(self, title: str, logo_path: Optional[Path], footer_text: str):
            super().__init__(orientation="P", unit="mm", format="A4")
            self.doc_title = title
            self.logo_path = logo_path
            self.footer_text = footer_text

        def header(self):
            y = 8
            if self.logo_path and Path(self.logo_path).exists():
                from PIL import Image

                with Image.open(self.logo_path) as im:
                    ratio = im.height / im.width
                w = 42
                h = w * ratio
                self.image(str(self.logo_path), x=(self.w - w) / 2, y=y, w=w)
                self.set_y(y + h + 2)
            else:
                self.set_y(y)
                self.set_font("Helvetica", "B", 16)
                self.cell(0, 9, "LegalEase", align="C", new_x=XPos.LMARGIN, new_y=YPos.NEXT)
            self.set_font("Helvetica", "B", 12)
            self.cell(0, 7, _pdf_safe(self.doc_title), align="C", new_x=XPos.LMARGIN,
    new_y=YPos.NEXT)
            self.ln(3)

        def footer(self):
            self.set_y(-15)
            self.set_font("Helvetica", "I", 8)
            self.cell(0, 10, _pdf_safe(f"{self.footer_text} | Page {self.page_no()} of {{nb}}"),
    align="C")

    return LegalPDF, XPos, YPos
def format_pdf(
    text: str,
    doc_type: str = "Legal Document",
    terms: str = "",
    parties: str = "",
    effective_date: str = "",
) -> bytes:
    """Build a branded PDF: centred logo + title in the header, footer on every page."""
    LegalPDF, XPos, YPos = _make_pdf_class()
    title, blocks = parse_document(text, doc_type)

    pdf = LegalPDF(title, Path(config.DOC_LOGO_PATH), config.FOOTER_TEXT)
    pdf.set_title(_pdf_safe(title))
    pdf.set_author(config.COMPANY_NAME)
    pdf.set_margins(20, 20, 20)
    pdf.set_auto_page_break(auto=True, margin=22)
    pdf.alias_nb_pages()
    pdf.add_page()

    def write(txt, style="", size=11, align="L", indent=0.0, h=6.0, gap=0.0):
        pdf.set_font("Helvetica", style, size)
        pdf.set_x(pdf.l_margin + indent)
        pdf.multi_cell(0, h, _pdf_safe(txt), align=align, new_x=XPos.LMARGIN, new_y=YPos.NEXT)
        if gap:
```

```
            pdf.ln(gap)

    details = _detail_rows(doc_type, parties, effective_date)
    for label, value in details:
        write(f"{label}: {value}", size=10, h=5.5)
    term_items = split_terms(terms)
    if term_items:
        pdf.ln(2)
        write("Terms & Conditions", style="B", size=11, gap=1)
        for t in term_items:
            write(f"-  {t}", size=10.5, indent=4, h=5.5)
    if details or term_items:
        pdf.ln(3)
        pdf.line(pdf.l_margin, pdf.get_y(), pdf.w - pdf.r_margin, pdf.get_y())
        pdf.ln(5)

    for kind, txt in blocks:
        if kind == "heading":
            pdf.ln(2)
            write(txt, style="B", size=11.5, gap=1)
        elif kind == "bullet":
            write(f"-  {txt}", indent=6, h=5.8, gap=0.8)
        elif kind == "sub":
            write(txt, indent=8, h=5.8, gap=0.8)
        elif kind == "rule":
            pdf.ln(9)
            y = pdf.get_y()
            pdf.line(pdf.l_margin, y, pdf.l_margin + 65, y)
            pdf.ln(2)
        else:
            write(txt, align="J", h=6, gap=2)

    out = pdf.output()
    return bytes(out) if isinstance(out, (bytes, bytearray)) else str(out).encode("latin-1")
```

ai_core/text_utils.py

```
"""Small, dependency-free text helpers shared by the frontend and exporters."""
import re
from typing import List

_REPLACEMENTS = {
    "\u2018": "'", "\u2019": "'", "\u201a": "'", "\u201b": "'",
    "\u201c": '"', "\u201d": '"', "\u201e": '"',
    "\u2013": "-", "\u2014": "-", "\u2212": "-",
    "\u2026": "...", "\u2192": "->",
    "\u2022": "-", "\u25cf": "-", "\u25aa": "-", "\u25e6": "-",
    "\u00a0": " ", "\u2002": " ", "\u2003": " ", "\u2009": " ",
    "\u200b": "", "\ufeff": "",
}

_CONTROL_CHARS = re.compile(r"[\x00-\x08\x0b\x0c\x0e-\x1f\x7f]")


def sanitize_text(text) -> str:
    """Remove typographic quotes, markdown artefacts and control characters.

    The result is clean plain text that is safe for .txt, .docx and .pdf output.
    """
    if not text:
        return ""
    text = str(text).replace("\r\n", "\n").replace("\r", "\n").replace("\t", "    ")
    for old, new in _REPLACEMENTS.items():
        text = text.replace(old, new)
    text = _CONTROL_CHARS.sub("", text)

    text = re.sub(r"```[a-zA-Z]*\n?", "", text)                      # code fences
    text = re.sub(r"(?m)^\s*(?:-{3,}|\*{3,})\s*$", "", text)        # horizontal rules
```

```
    text = re.sub(r"(?m)^\s{0,3}#{1,6}\s*", "", text)                # '## Heading'
    text = re.sub(r"(?m)^(\s*)\*\s+", r"\1- ", text)                 # '* item' -> '- item'
    text = re.sub(r"\*\*([^\n]+?)\*\*", r"\1", text)                 # **bold**
    text = re.sub(r"__([^_\n]+?)__", r"\1", text)                    # __bold__ (keeps ____
    signature lines)

    text = re.sub(r"[ ]+\n", "\n", text)
    text = re.sub(r"\n{3,}", "\n\n", text)
    return text.strip()


def split_terms(terms: str) -> List[str]:
    """Split semicolon / newline separated terms into a clean list."""
    if not terms:
        return []
    items = []
    for part in re.split(r"[;\n]+", str(terms)):
        part = re.sub(r"^\s*(?:[-*]|\d+[.)])\s+", "", part).strip()
        if part:
            items.append(part)
    return items


def safe_filename(name: str, default: str = "legal_document") -> str:
    slug = re.sub(r"[^a-z0-9]+", "_", (name or "").lower()).strip("_")
    return slug[:60] or default
```

frontend/app.py

```
"""LegalEase - Streamlit frontend."""
import sys
from pathlib import Path

# Make the project root importable (Streamlit only adds this file's folder).
ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

import requests  # noqa: E402
import streamlit as st  # noqa: E402

import config  # noqa: E402
from ai_core.generator import (  # noqa: E402
    format_docx,
    format_html_preview,
    format_pdf,
    safe_filename,
    sanitize_text,
)

st.set_page_config(page_title="LegalEase", page_icon="??", layout="centered")

# ---------------------------------------------------------------------- state
for key, default in {"generated_text": "", "meta": {}, "show_edit": False, "just_generated":
    False}.items():
    st.session_state.setdefault(key, default)


def open_editor():
    st.session_state.editor_text = st.session_state.generated_text
    st.session_state.show_edit = True


def close_editor():
    if "editor_text" in st.session_state:
        st.session_state.generated_text = st.session_state.editor_text
    st.session_state.show_edit = False
def backend_health():
```

```
    try:
        r = requests.get(f"{config.BACKEND_URL}/health", timeout=3)
        return r.json() if r.ok else None
    except Exception:
        return None


def error_detail(resp: requests.Response) -> str:
    try:
        detail = resp.json().get("detail", resp.text)
    except ValueError:
        detail = resp.text
    if isinstance(detail, list):  # FastAPI validation errors
        detail = "; ".join(f"{'.'.join(map(str, d.get('loc', [])[1:]))}: {d.get('msg')}" for d in
    detail)
    return str(detail)


# ---------------------------------------------------------------- sidebar
with st.sidebar:
    st.subheader("Backend status")
    health = backend_health()
    if health is None:
        st.error("Backend offline")
        st.caption(f"Start it with:\n\n`uvicorn legalEaseAPI.main:app --reload`\n\nExpected at
    {config.BACKEND_URL}")
    else:
        st.success("Backend online")
        st.caption(f"Model: `{health.get('model')}`")
        if not health.get("api_key_configured"):
            st.warning("GEMINI_API_KEY is not set in .env")
    st.divider()
    st.caption("AI-generated drafts are not legal advice. Have a qualified lawyer review any
    document before signing.")

# ---------------------------------------------------------------- header
_, mid, _ = st.columns([1, 2, 1])
with mid:
    if Path(config.WEB_LOGO_PATH).exists():
        st.image(str(config.WEB_LOGO_PATH), width=260)
    else:
        st.markdown("<h1 style='text-align:center;'>?? LegalEase</h1>", unsafe_allow_html=True)

st.markdown("<h2 style='text-align: center;'>AI Legal Document Generator</h2>",
    unsafe_allow_html=True)

# ---------------------------------------------------------------- inputs
document_type = st.text_input("Document Type (Ex: Agreement, Contract, NDA)", placeholder="Freelance
     Work Contract")
parties = st.text_area("Parties Involved", placeholder="Jane Doe (Service Provider), TechNova Inc.
    (Client)")
terms = st.text_area(
    "Terms & Conditions (Use semicolons for bullet points)",
    placeholder="Work must be delivered by May 15, 2025; Payment within 7 days of invoice;
    Confidentiality at all times",
)
dates = st.text_input("Effective Date", placeholder="April 15, 2025")

if st.button("Generate Document", type="primary"):
    missing = [n for n, v in (("Document Type", document_type), ("Parties Involved", parties),
                              ("Effective Date", dates)) if not v.strip()]
    if missing:
        st.error("Please fill in: " + ", ".join(missing))
    else:
        payload = {"document_type": document_type, "parties": parties, "terms": terms, "dates":
    dates}
        try:
            with st.spinner("Drafting your document with Gemini... this can take up to a minute."):
                response = requests.post(f"{config.BACKEND_URL}/generate", json=payload,
    timeout=config.REQUEST_TIMEOUT)
```

```
            if response.status_code == 200:
                st.session_state.generated_text = sanitize_text(response.json()["document"])
                st.session_state.meta = {
                    "document_type": document_type.strip(),
                    "parties": parties.strip(),
                    "terms": terms.strip(),
                    "dates": dates.strip(),
                }
                st.session_state.show_edit = False
                st.session_state.pop("editor_text", None)
                st.session_state.just_generated = True
            else:
                st.error(f"Generation failed ({response.status_code}): {error_detail(response)}")
        except requests.exceptions.ConnectionError:
            st.error(f"Cannot reach the backend at {config.BACKEND_URL}. Start it with: uvicorn
    legalEaseAPI.main:app --reload")
        except requests.exceptions.Timeout:
            st.error("The request timed out. Please try again.")
        except Exception as exc:  # noqa: BLE001
            st.error(f"Unexpected error: {exc}")

# ------------------------------------------------------------------ results
if not st.session_state.generated_text:
    st.info("Click 'Generate Document' to start")
else:
    # Pick up edits made in the text area (widget state is current at rerun start).
    if st.session_state.show_edit and "editor_text" in st.session_state:
        st.session_state.generated_text = st.session_state.editor_text
    text = st.session_state.generated_text
    meta = st.session_state.meta

    if st.session_state.just_generated:
        st.success("? Document Generated Successfully!")
        st.session_state.just_generated = False

    card = (
        "<div style='background:#0f172a;border:1px solid #1e293b;border-radius:10px;padding:22px
    26px;"
        "max-height:440px;overflow-y:auto;color:#e2e8f0;font-family:Georgia,\"Times New
    Roman\",serif;"
        f"font-size:0.95rem;line-height:1.6;'>{format_html_preview(text, meta.get('document_type',
    ''))}</div>"
    )
    st.markdown(card, unsafe_allow_html=True)

    if st.session_state.show_edit:
        st.text_area("Edit Document Below:", key="editor_text", height=350)
        st.button("?? Done Editing", on_click=close_editor)
    else:
        st.button("?? Click to Edit Document", on_click=open_editor)

    base = safe_filename(meta.get("document_type", ""))
    doc_kwargs = dict(
        doc_type=meta.get("document_type") or "Legal Document",
        terms=meta.get("terms", ""),
        parties=meta.get("parties", ""),
        effective_date=meta.get("dates", ""),
    )

    st.download_button("? Download as .TXT", data=text.encode("utf-8"), file_name=f"{base}.txt",
    mime="text/plain")

    try:
        st.download_button(
            "? Download as .DOCX",
            data=format_docx(text, **doc_kwargs),
            file_name=f"{base}.docx",
            mime="application/vnd.openxmlformats-officedocument.wordprocessingml.document",
        )
    except Exception as exc:  # noqa: BLE001
```

```
        st.error(f"Could not build the DOCX file: {exc}")

    try:
        st.download_button(
            "? Download as .PDF",
            data=format_pdf(text, **doc_kwargs),
            file_name=f"{base}.pdf",
            mime="application/pdf",
        )
    except Exception as exc:  # noqa: BLE001
        st.error(f"Could not build the PDF file: {exc}")
```

legalEaseAPI/__init__.py

legalEaseAPI/main.py

```
import logging
import sys
from pathlib import Path

# Make the project root importable even when this file is run directly.
ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from fastapi import FastAPI  # noqa: E402

from legalEaseAPI.routes import router  # noqa: E402

logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(name)s: %(message)s")

app = FastAPI(title="LegalEase - AI Legal Document Generator", version="1.0.0")

# Include API routes
app.include_router(router)


# Root endpoint
@app.get("/")
def home():
    return {"message": "Welcome to LegalEase AI Legal Document Generator API"}


# Run FastAPI if executed directly
if __name__ == "__main__":
    import uvicorn

    uvicorn.run(app, host="0.0.0.0", port=8000)
```

legalEaseAPI/routes.py

```
import logging

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel, ConfigDict, Field

from ai_core.gemini_generator import (
    GeminiConfigError,
    GeminiDocumentGenerator,
    GeminiGenerationError,
)

logger = logging.getLogger(__name__)
```

```
router = APIRouter()
gemini_generator = GeminiDocumentGenerator()


class DocumentRequest(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True)

    document_type: str = Field(..., min_length=2, max_length=200)
    parties: str = Field(..., min_length=2, max_length=2000)
    terms: str = Field("", max_length=10000)
    dates: str = Field(..., min_length=2, max_length=100)


@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    try:
        response = gemini_generator.generate_document(
            request.document_type,
            request.parties,
            request.terms,
            request.dates,
        )
    except GeminiConfigError as exc:
        logger.error("Configuration error: %s", exc)
        raise HTTPException(status_code=500, detail=str(exc)) from exc
    except GeminiGenerationError as exc:
        logger.error("Generation error: %s", exc)
        raise HTTPException(status_code=502, detail=str(exc)) from exc
    return {"document": response, "model": gemini_generator.active_model}


@router.get("/health")
def health():
    return {
        "status": "ok",
        "api_key_configured": bool(gemini_generator.api_key)
        and not gemini_generator.api_key.startswith("your_"),
        "model": gemini_generator.active_model or gemini_generator.model_name,
    }
```

scripts/check_gemini.py

```
"""Quick live check of your API key and model.  Run:  python scripts/check_gemini.py"""
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from ai_core.gemini_generator import GeminiConfigError, GeminiDocumentGenerator,
    GeminiGenerationError  # noqa: E402

gen = GeminiDocumentGenerator()
try:
    text = gen.generate_document("Mutual NDA", "Alice (Discloser), Bob Ltd (Recipient)",
                                 "Keep information confidential; Term of 2 years", "January 1,
    2026")
except (GeminiConfigError, GeminiGenerationError) as exc:
    print("FAILED:", exc)
    sys.exit(1)
print(f"OK - model '{gen.active_model}' produced {len(text)} characters.\n")
print(text[:600], "...")
```

scripts/create_logos.py

```
"""Generates Image/Logo.png (dark, for documents) and Image/inverseLogo.png (light, for the dark web
     UI).
```

```
Replace those two files with your own branding at any time; the app only needs
the two PNG files to exist.  Run:  python scripts/create_logos.py
"""
from pathlib import Path

from PIL import Image, ImageDraw, ImageFont

OUT = Path(__file__).resolve().parents[1] / "Image"
FONT_CANDIDATES = [
    "C:/Windows/Fonts/times.ttf",
    "/System/Library/Fonts/Supplemental/Times New Roman.ttf",
    "/usr/share/fonts/truetype/dejavu/DejaVuSerif.ttf",
    "/usr/share/fonts/truetype/liberation/LiberationSerif-Regular.ttf",
    "DejaVuSerif.ttf",
]


def load_font(size: int):
    for path in FONT_CANDIDATES:
        try:
            return ImageFont.truetype(path, size)
        except OSError:
            continue
    return ImageFont.load_default(size)


def draw_logo(color) -> Image.Image:
    img = Image.new("RGBA", (1100, 300), (0, 0, 0, 0))
    d = ImageDraw.Draw(img)
    w = 9
    d.line((130, 60, 130, 205), fill=color, width=w)          # pole
    d.line((85, 210, 175, 210), fill=color, width=w)          # base
    d.line((40, 85, 220, 85), fill=color, width=w)            # beam
    for cx in (55, 205):                                      # pans
        d.line((cx, 85, cx - 30, 150), fill=color, width=6)
        d.line((cx, 85, cx + 30, 150), fill=color, width=6)
        d.arc((cx - 32, 118, cx + 32, 182), 0, 180, fill=color, width=6)
    d.ellipse((120, 50, 140, 70), fill=color)
    d.text((280, 135), "LegalEase", font=load_font(130), fill=color, anchor="lm")
    bbox = img.getbbox()
    pad = 16
    return img.crop((bbox[0] - pad, bbox[1] - pad, bbox[2] + pad, bbox[3] + pad))
if __name__ == "__main__":
    OUT.mkdir(exist_ok=True)
    draw_logo((20, 20, 20, 255)).save(OUT / "Logo.png")
    draw_logo((245, 245, 245, 255)).save(OUT / "inverseLogo.png")
    print("Logos written to", OUT)
```

tests/test_api.py

```
import pytest

pytest.importorskip("fastapi")
pytest.importorskip("httpx")

from fastapi.testclient import TestClient  # noqa: E402

from ai_core.gemini_generator import GeminiConfigError, GeminiGenerationError  # noqa: E402
from legalEaseAPI import routes  # noqa: E402
from legalEaseAPI.main import app  # noqa: E402

client = TestClient(app)
PAYLOAD = {"document_type": "NDA", "parties": "A (Disclosing), B (Receiving)", "terms": "Keep
    secret; 2 years", "dates": "May 1, 2026"}


def test_root_and_health():
    assert "Welcome" in client.get("/").json()["message"]
```

```
    assert client.get("/health").json()["status"] == "ok"


def test_generate_success(monkeypatch):
    monkeypatch.setattr(routes.gemini_generator, "generate_document", lambda *a: "THE DOCUMENT")
    r = client.post("/generate", json=PAYLOAD)
    assert r.status_code == 200 and r.json()["document"] == "THE DOCUMENT"


def test_generate_validation_error():
    r = client.post("/generate", json={**PAYLOAD, "document_type": ""})
    assert r.status_code == 422


def test_generate_error_mapping(monkeypatch):
    def cfg(*a):
        raise GeminiConfigError("no key")

    def gen(*a):
        raise GeminiGenerationError("model down")

    monkeypatch.setattr(routes.gemini_generator, "generate_document", cfg)
    assert client.post("/generate", json=PAYLOAD).status_code == 500
    monkeypatch.setattr(routes.gemini_generator, "generate_document", gen)
    assert client.post("/generate", json=PAYLOAD).status_code == 502
```

tests/test_formatting.py

```
import io
import zipfile

import pytest

from ai_core.generator import (
    format_docx,
    format_html_preview,
    format_pdf,
    parse_document,
    safe_filename,
    sanitize_text,
    split_terms,
)

SAMPLE = """## Freelance Work Contract

Agreement made this 15th day of April, 2025

Between:

Jane Doe (hereinafter the \u201cService Provider\u201d) and TechNova Inc. (the \u201cClient\u201d).

1. Services:

The Service Provider shall deliver the work by May 15, 2025 for a fee of $5,000.

2. Term and Termination:

a) Mutual written agreement of the parties;
b) Breach of this Agreement by either party.

- Confidentiality must be maintained
- Payment within 7 days

IN WITNESS WHEREOF, the parties have executed this Agreement.

_____
Jane Doe (Service Provider)
"""
```

```
def test_sanitize_removes_markdown_and_smart_quotes():
    out = sanitize_text("## Title\n**bold** \u201cquoted\u201d \u2014 dash\n* item")
    assert "#" not in out and "**" not in out
    assert '"quoted"' in out and "-" in out
    assert out.splitlines()[-1] == "- item"


def test_sanitize_keeps_signature_lines():
    assert "_____________" in sanitize_text("_____________")


def test_split_terms_and_filename():
    assert split_terms("A; B ;\n- C;;") == ["A", "B", "C"]
    assert safe_filename("Freelance Work Contract!") == "freelance_work_contract"
    assert safe_filename("???") == "legal_document"


def test_parse_document_kinds():
    title, blocks = parse_document(SAMPLE, "Freelance Work Contract")
    assert title == "Freelance Work Contract"
    kinds = dict((t, k) for k, t in blocks)
    assert kinds["1. Services:"] == "heading"
    assert kinds["Between:"] == "heading"
    assert kinds["a) Mutual written agreement of the parties;"] == "sub"
    assert kinds["Confidentiality must be maintained"] == "bullet"
    assert kinds["_______________________________"] == "rule"
    assert kinds["Jane Doe (Service Provider)"] == "para"


def test_html_preview_is_escaped_and_single_block():
    html_out = format_html_preview("Title\n\n<script>alert(1)</script> costs $5 and $6", "Title")
    assert "<script>" not in html_out
    assert "$" not in html_out and "&#36;" in html_out
    assert "\n" not in html_out


def test_docx_roundtrip():
    from docx import Document

    data = format_docx(SAMPLE, "Freelance Work Contract", terms="Pay in 7 days; Deliver by May 15",
                       parties="Jane Doe, TechNova", effective_date="April 15, 2025")
    assert zipfile.is_zipfile(io.BytesIO(data))
    doc = Document(io.BytesIO(data))
    text = "\n".join(p.text for p in doc.paragraphs)
    assert "Freelance Work Contract" in text
    assert "1. Services:" in text
    assert len(doc.tables) == 2                       # details + terms tables
    assert "Pay in 7 days" in doc.tables[1].rows[1].cells[1].text
    assert "All Rights Reserved" in doc.sections[0].footer.paragraphs[0].text


def test_pdf_is_valid():
    pytest.importorskip("fpdf")
    data = format_pdf(SAMPLE * 6, "Freelance Work Contract", terms="A; B", parties="X, Y",
    effective_date="today")
    assert data.startswith(b"%PDF")
    assert len(data) > 2000


def test_pdf_handles_non_latin_text():
    pytest.importorskip("fpdf")
    assert format_pdf("Rent is \u20b95,000 per month \u2705", "Lease Agreement").startswith(b"%PDF")
```

tests/test_gemini_generator.py

```
import pytest

from ai_core import gemini_generator as gg
```

```
def make(monkeypatch, behaviours, **kw):
    gen = gg.GeminiDocumentGenerator(model_name="m1", api_key="real-key", fallback_models=["m2",
    "m3"], **kw)
    monkeypatch.setattr(gen, "_ensure_client", lambda: None)
    calls = []

    def fake_call(model, prompt):
        calls.append(model)
        result = behaviours[model]
        if isinstance(result, Exception):
            raise result
        return result

    monkeypatch.setattr(gen, "_call_model", fake_call)
    return gen, calls


def test_prompt_contains_inputs_and_numbered_terms():
    p = gg.build_prompt("NDA", "A and B", "One; Two", "May 1, 2026")
    assert "'NDA'" in p and "A and B" in p and "May 1, 2026" in p
    assert "1. One" in p and "2. Two" in p


def test_falls_back_to_next_model(monkeypatch):
    gen, calls = make(monkeypatch, {"m1": RuntimeError("404 NOT_FOUND model"), "m2": "DOC", "m3":
    "x"})
    assert gen.generate_document("NDA", "A", "", "today") == "DOC"
    assert calls == ["m1", "m2"] and gen.active_model == "m2"
    gen.generate_document("NDA", "A", "", "today")
    assert calls[2] == "m2"          # remembers the working model


def test_auth_error_is_not_retried(monkeypatch):
    gen, calls = make(monkeypatch, {"m1": RuntimeError("API key not valid. Please pass a valid API
    key."), "m2": "x", "m3": "x"})
    with pytest.raises(gg.GeminiConfigError):
        gen.generate_document("NDA", "A", "", "today")
    assert calls == ["m1"]
def test_all_models_fail(monkeypatch):
    err = RuntimeError("boom")
    gen, _ = make(monkeypatch, {"m1": err, "m2": err, "m3": err})
    with pytest.raises(gg.GeminiGenerationError):
        gen.generate_document("NDA", "A", "", "today")


def test_missing_key_raises_config_error():
    gen = gg.GeminiDocumentGenerator(api_key="", model_name="m1")
    with pytest.raises(gg.GeminiConfigError):
        gen.generate_document("NDA", "A", "", "today")
```